Introduction

Creational Patterns

When it comes to software design, we have some popular design patterns that provide the solution to globally occurring design problems. One family of such patterns is known as "Creational Patterns". These patterns help us create objects so that the system remains flexible and is not tied to specific object details.

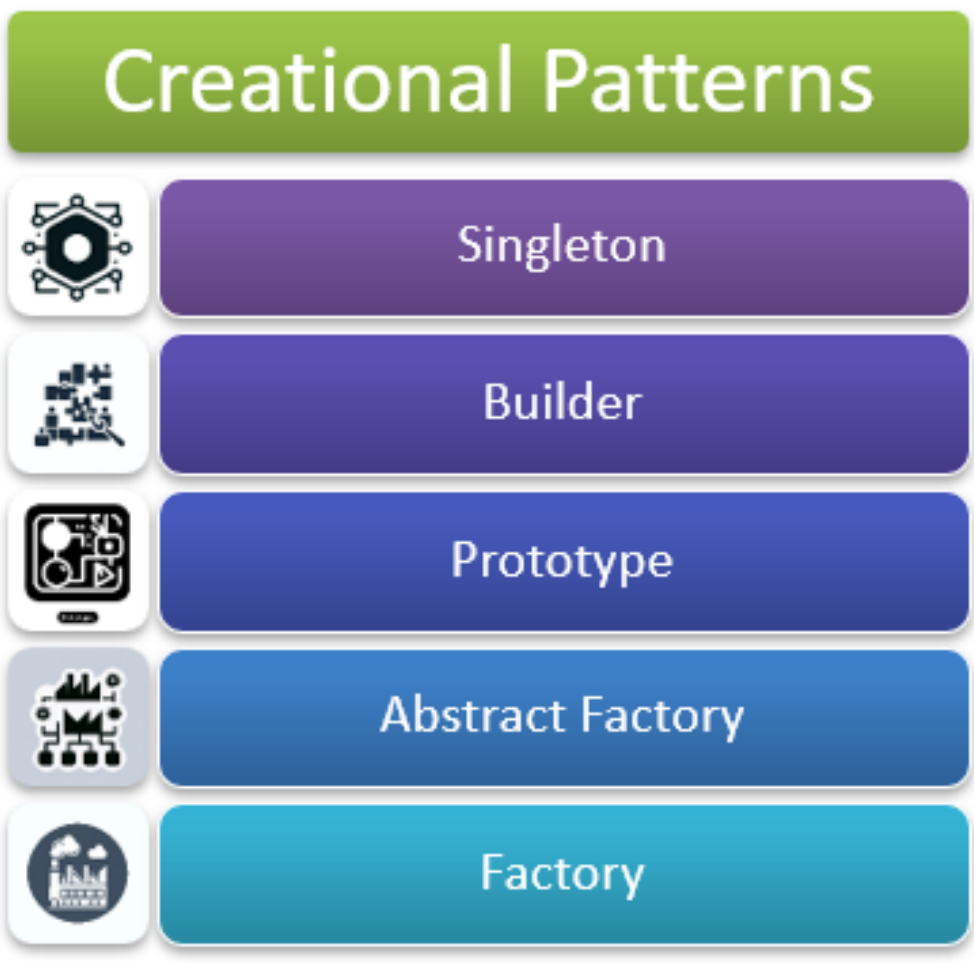
Creational Patterns focus on how we create objects without revealing the technical details of these objects. This means they hide the technical stuff in creating objects and choosing their types.

Adopting creational patterns basically guarantees that your system can:

- Be independent of the creation and composition of its objects.
- Hiding the complexity of creation and representation.
- Allow for some freedom regarding what gets created, who creates it, and how.

Common Creational Patterns

Some of the commonly used creational patterns are:



Common Creational Patterns

- **Singleton Pattern**: This type of pattern ensures that there is a single instance of a specific class and provides global access to this instance.
- **Builder Pattern**: This type of pattern separates the construction of complex objects from their representations. This allows the same construction process for similar kinds of objects.
- **Prototype Type**: This pattern is used when a prototypical instance specifies the kind of objects to create, which is then cloned to produce new objects.
- **Factory Pattern**: This pattern provides an instance for creating an instance of a class, with its subclasses to decide which class object to initiate.
- **Abstract Factory Pattern**: This pattern allows the creation of object families, which are related or dependent on each other, without specifying concrete classes.